HawkEye Threat Model Database Architecture Report

Cyber Security Technologies (ITMS 448/548)

Executive Summary

The HawkEye OSINT Analysis Tool is an open-source Python desktop application designed to analyze whether images shared in Reddit news posts are being used in their correct context. The application combines reverse image search, metadata extraction, article scraping, fact-check APIs, and local AI analysis to determine whether an image is consistent with the event it claims to represent.

This report focuses on the HawkEye threat model and database architecture developed using the Microsoft Threat Modeling Tool. The architecture identifies the major system components, trust boundaries, data flows, external dependencies, and security risks associated with the HawkEye pipeline. The threat model applies STRIDE-based analysis to evaluate risks including spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.

The threat modeling process identified potential threats across the system architecture, with the majority categorized as High priority. Major risks include SQL injection, insecure configuration storage, credential theft, spoofed external services, information leakage through logs or error messages, weak encryption, and insufficient auditing controls. Mitigation strategies were proposed for each identified threat to strengthen the system's overall security posture.

1. Introduction

HawkEye is a desktop-based Open Source Intelligence (OSINT) analysis application developed in Python. The application is designed to determine whether a Reddit image post is being reused out of context by comparing the image against historical appearances and fact-check evidence.

The system accepts a Reddit URL or image-and-claim pair, retrieves associated content, performs reverse image searching, extracts metadata, gathers historical article evidence, and generates a verdict using a locally hosted large language model through Ollama. The final analysis is displayed through a graphical user interface and can be exported as JSON or PDF reports.

The purpose of this report is to document the HawkEye threat model and database architecture, identify major system threats, analyze security risks, and describe mitigation strategies used to secure the application.

2. System Architecture Overview

The HawkEye architecture is divided into multiple functional layers:

- User Interface Layer
- Orchestration Layer
- Intelligence Layer
- Data Access Layer
- Storage and Export Layer
- External Data Source Layer

Major Components

PySide6 GUI

The graphical user interface allows users to submit Reddit URLs or image inputs and view analysis results. The GUI also supports exporting reports as PDF or JSON files.

HawkEye Orchestrator

The orchestrator acts as the core processing engine. It coordinates:

- Metadata extraction
- Reverse image searching
- Article retrieval
- Fact-check analysis
- LLM verdict generation
- Cache retrieval
- Report generation

EXIFTool Metadata Extraction

- The system uses EXIFTool locally to retrieve metadata from downloaded images, including timestamps, dimensions, file types, and available GPS information.

Local LLM / Ollama

- The intelligence layer uses a locally hosted AI model through Ollama to analyze claims and historical evidence. This allows HawkEye to avoid transmitting sensitive content to cloud-hosted AI services.

Temporary JSON Storage / Cache

- A temporary storage system stores cached analysis results keyed by image perceptual hashes to avoid repeated analysis and unnecessary API calls.

External Data Sources

The architecture communicates with several external services:

- SerpAPI / TinEye
- News Websites / Article Sources
- Google Fact Check API
- User-provided URLs and Reddit posts

3. Trust Boundaries

The HawkEye architecture contains multiple trust boundaries separating internal and external components.

Internal HawkEye System

This trust boundary contains:

- HawkEye Orchestrator
- GUI
- Local LLM
- Cache storage
- EXIFTool
- Report export functionality

External Data Sources

This trust boundary includes:

- News article websites
- SerpAPI
- TinEye
- Google Fact Check API
- User-provided URLs

Because these services operate outside of the local environment, they introduce higher security risk and reduced visibility.

4. Data Flow Analysis

The HawkEye pipeline operates through several major data flows:

1. User submits Reddit URL or image input
2. GUI sends analysis request to the HawkEye Orchestrator
3. Image is processed through EXIFTool
4. Reverse image search is performed using SerpAPI or TinEye
5. Article content is scraped from discovered URLs
6. Claim text is sent to Google Fact Check API
7. Historical evidence and claim text are processed by the local LLM
8. Verdict and reasoning are generated
9. Results are cached and exported as JSON/PDF reports

The orchestrator acts as the primary communication hub between all internal and external services.

5. Threat Modeling Methodology

The Microsoft Threat Modeling Tool was used to identify security risks using the STRIDE methodology.

The STRIDE categories include:

- Spoofing
- Tampering
- Repudiation
- Information Disclosure
- Denial of Service
- Elevation of Privilege

The HawkEye threat model identified 98 total threats within the system architecture. Most identified threats were classified as High priority.

6. Major Threats Identified

6.1 Spoofing Threats

Spoofing threats involve attackers impersonating trusted services or components.

Examples

- Spoofed News Article Sources
- Spoofed Google Fact Check API
- Spoofed HawkEye Orchestrator
- Fake websites used for phishing attacks
- Insecure TLS certificate configuration

Risk

Attackers may redirect the system toward malicious websites or inject false evidence into the analysis pipeline.

Mitigations

- TLS certificate validation
- X.509 certificate verification
- Standard authentication mechanisms
- Safe redirect validation
- Secure API communication

6.2 Tampering Threats

Tampering threats involve unauthorized modification of application data or database content.

Examples

- SQL injection attacks
- Malicious modification of cached analysis results
- Tampering with configuration files
- Manipulation of report exports

Risk

Attackers could alter analysis outcomes, inject malicious queries, or compromise stored system data.

Mitigations

- Parameterized SQL queries
- Input validation
- Secure configuration storage
- Database auditing
- Restricted permissions

6.3 Repudiation Threats

Repudiation threats occur when users or attackers deny actions due to insufficient logging.

Examples

- Lack of auditing
- Missing security logs
- Inadequate tracking of analysis requests

Risk

Without proper logs, malicious actions may become difficult to investigate.

Mitigations

- Centralized logging
- Audit log rotation
- Restricted access to logs
- User action tracking

6.4 Information Disclosure Threats

Information disclosure threats expose sensitive information to unauthorized parties.

Examples

- Verbose error messages
- Sensitive log files
- Weak encryption
- Leaked credentials
- Unencrypted database communication

Risk

Attackers could obtain API keys, system paths, stack traces, or cached analysis data.

Mitigations

- Secure error handling
- Encrypted communication
- Hashing and encryption standards
- Hidden configuration secrets
- Restricted log access

6.5 Elevation of Privilege Threats

Elevation of privilege occurs when attackers gain permissions beyond intended access levels.

Examples

- Loose authorization rules
- Weak database access controls
- Unrestricted firewall access

Risk

Attackers may gain administrative access to the system or database.

Mitigations

- Least privilege access
- Firewall protections
- Role-based authorization
- Restricted database access

7. Security Controls Implemented

The HawkEye architecture already includes several security-oriented design decisions:

- Local AI inference using Ollama
- Gitignored configuration files
- Local-only cache storage
- No cloud-hosted database
- Dependency version pinning
- Protected GitHub main branch
- Pull request review requirements
- Temporary local JSON caching

These controls help reduce exposure of sensitive information and minimize cloud-based attack surfaces.

8. Testing and Validation

The HawkEye project includes multiple testing scenarios covering:

- GUI functionality
- EXIF metadata extraction
- Reverse image searching
- Article retrieval
- Archive history lookup
- URL safety analysis

Example successful tests include:

- EXIFTool extracting metadata fields from downloaded Reddit images
- Article scraping retrieving publication dates and article text
- Archive lookup retrieving historical snapshots
- URL safety analysis verifying domain reputation

9. Conclusion

The HawkEye threat model and database architecture provide a strong foundation for secure OSINT analysis. The architecture successfully separates trusted internal components from untrusted external services through clearly defined trust boundaries.

The Microsoft Threat Modeling Tool identified 98 potential threats across the system architecture, primarily involving spoofing, tampering, information disclosure, and privilege escalation. By applying STRIDE-based analysis and implementing mitigation strategies such as encryption, secure authentication, auditing, and input validation, the HawkEye project significantly improves its overall security posture.

In addition, since HawkEye processes external internet content and user-provided URLs, security remains a critical consideration throughout development. Continued testing, monitoring, and iterative security improvements will be essential as the system evolves.